

完整自学手册

GitHub 工作法

律师的协作新方式 · 零基础友好

← → 键翻页 | F 键全屏 | S 演讲者视图

为什么写这个手册



你可能是

看到那篇文章觉得有意思，但完全看不懂

被"版本混乱"困扰：最终版1/2/3

想让 AI 真正参与工作

团队越来越大，协作越来越乱



读完你能

理解 Git/GitHub 核心概念

用 Issue + PR 管理任务

让多人/多设备协作有章可循

知道什么该进 Git，什么不该进



阅读建议

每页先看"是什么"

再看"像什么"

最后"怎么做"

英文概念都有专题解释

目标：读完这个手册，你将能够用 GitHub 工作法管理法律项目，实现结构化协作。

三个场景，三种痛苦

场景一：版本地狱

答辩状最终版.docx

答辩状最终版2.docx

答辩状最终版2修改.docx

答辩状最终提交版.docx

对方律师反馈版.docx

问题：不敢删、不敢信

场景二：协作混乱

微信发了一段意见

邮件发了修改建议

飞书文档里批注

PDF 上标了密密麻麻

电话说了两句...

问题：意见散落各处

场景三：AI 用不起来

AI 能写能查

但结果怎么融入项目？

每次从零给上下文

无法审核、无法打回

AI 是临时工，不是项目成员

问题：无法持续协作

1 Git 是什么？

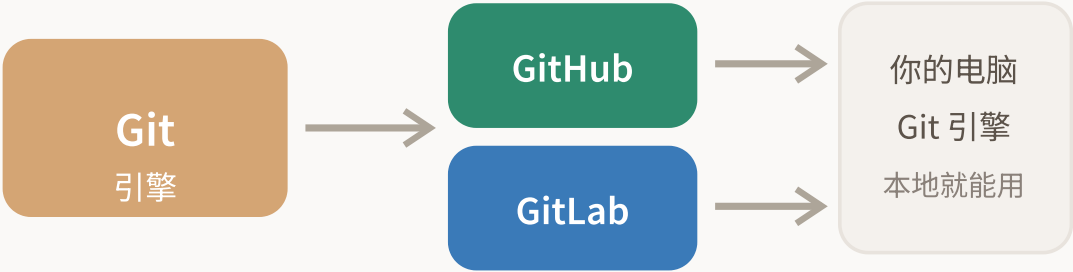
Git 是一个版本控制系统。记录项目每次变化，随时可以回到过去任意时刻。

像什么？

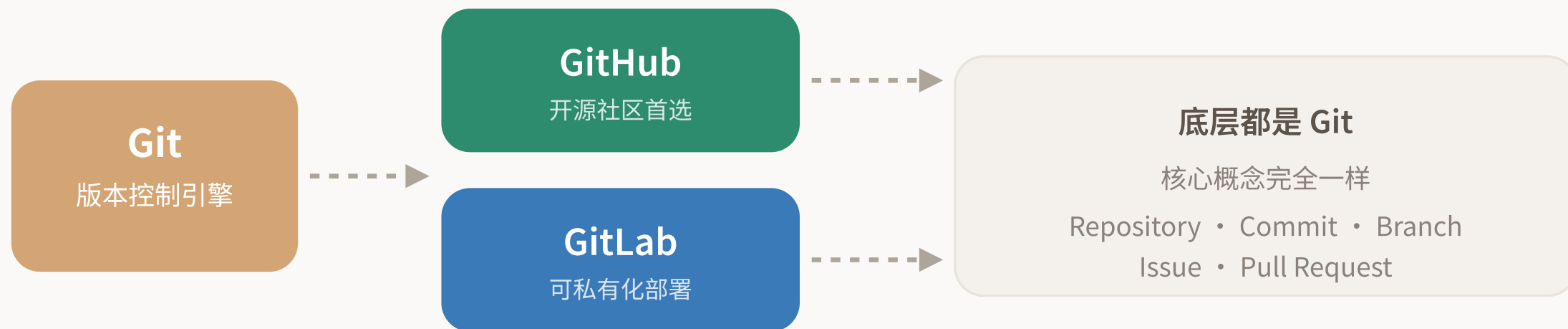
Git	生活中的类比
记录每次修改	写日记，每天发生什么都记下来
可以回到过去	游戏的存档点，死了可以读档重来
知道谁改的	账本，每笔支出都有记录和签名

关键理解

没有 GitHub，Git 照样工作。GitHub/GitLab 只是基于 Git 的云端平台，用于远程备份和多人协作。



Git 和 GitHub/GitLab 的关系



选择建议：团队内用 → GitLab（私有部署）；开源协作 → GitHub。底层都是 Git，概念互通。

2

Repository

仓库

Repository = 一个被 Git 管理的项目文件夹。中文常叫：仓库、项目仓库、代码库。

一个 Repository 包含



.git 文件夹

部位	功能
objects/	所有文件内容和 commit 快照
refs/	分支指针（指向哪个 commit）
HEAD	你现在在哪个分支
config	仓库配置

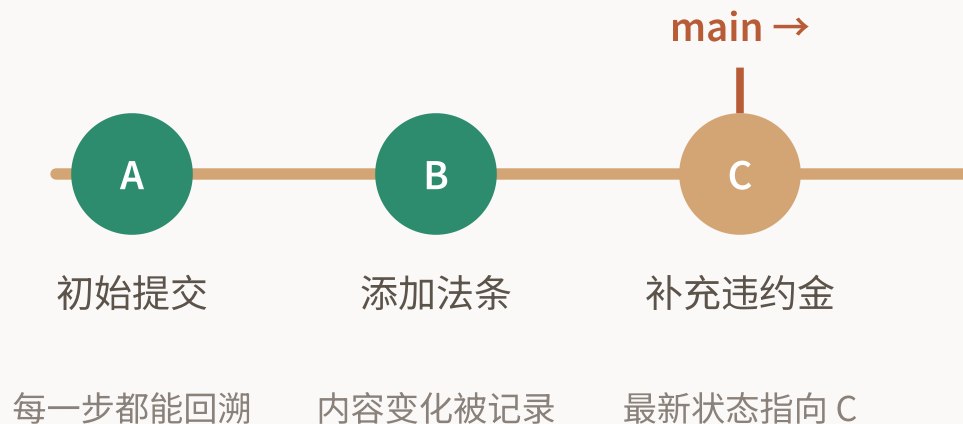
像什么？带自动存档功能的文件夹。改完可以存档，随时回到任意历史版本。

Commit = 给项目拍一张快照，永久保存。 不是"保存文件"，是"保存这一刻整个项目的完整状态"。

Commit 包含什么？

```
# Commit: a1b2c3d...
author: 张三
date: 2026-04-23 14:30
message: "补充违约金法条"
parent: 32b7fae...
tree: (当时所有文件的完整状态)
```

Commit 快照示意



像什么？ 游戏存档点。存了档，死了可以读档重来。

4

Branch

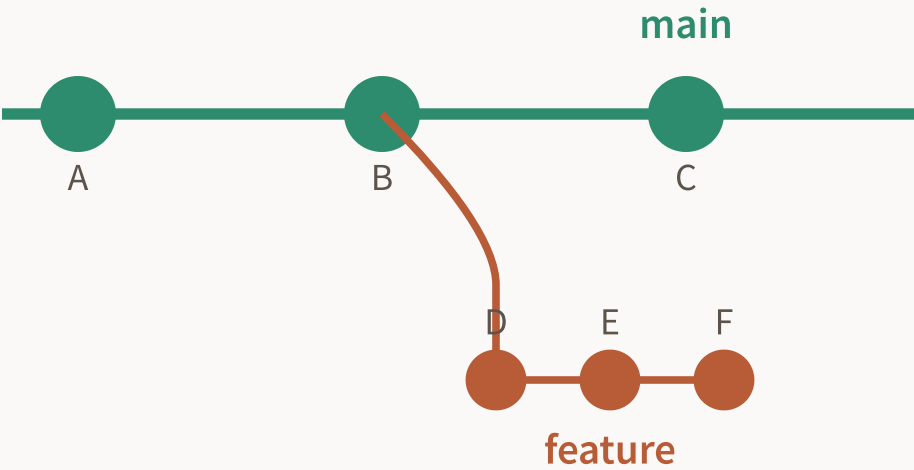
分支

Branch = 一个可移动的指针，指向某个 Commit。不是"复制文件"，是"多了一个指针"。

像什么？

书签	Branch
夹在书的某一页	指向某个 commit
可以移动到别的页	可以切换到别的 commit
多本书签互不干扰	多个分支并行开发
书本身只有一本	文件只有一份，指针有多个

分支示意



核心：切换分支 = 移动指针，工作目录文件跟着变。

git checkout = 切换分支，或恢复文件到某个状态。 两个完全不同的功能，同一个命令。

功能一：切换分支

```
git checkout main  
git checkout feature-1
```

Git 把 HEAD 指针移到目标分支，工作目录的文件跟着变。

像什么？ 换台——电视只有一个屏幕，换到频道1看新闻，换到频道2看体育。

功能二：恢复文件

```
git checkout -- 文件名  
git checkout a1b2c3d -- 文件名
```

放弃修改，恢复到最后一次提交。或恢复到某个 commit 的版本。

危险：切换分支时如果有未提交的修改，Git 会阻止切换。安全做法：先 `git add . + git commit`，再切换。

Worktree = 同一个仓库的额外工作目录。 让多个分支同时存在于不同文件夹。

Branch 分支

同一文件夹内切换

只能同时在一个分支上工作

切换时当前修改可能丢失

用 `git checkout` 切换

同一房间换桌子

Worktree 工作树

多个文件夹同时存在

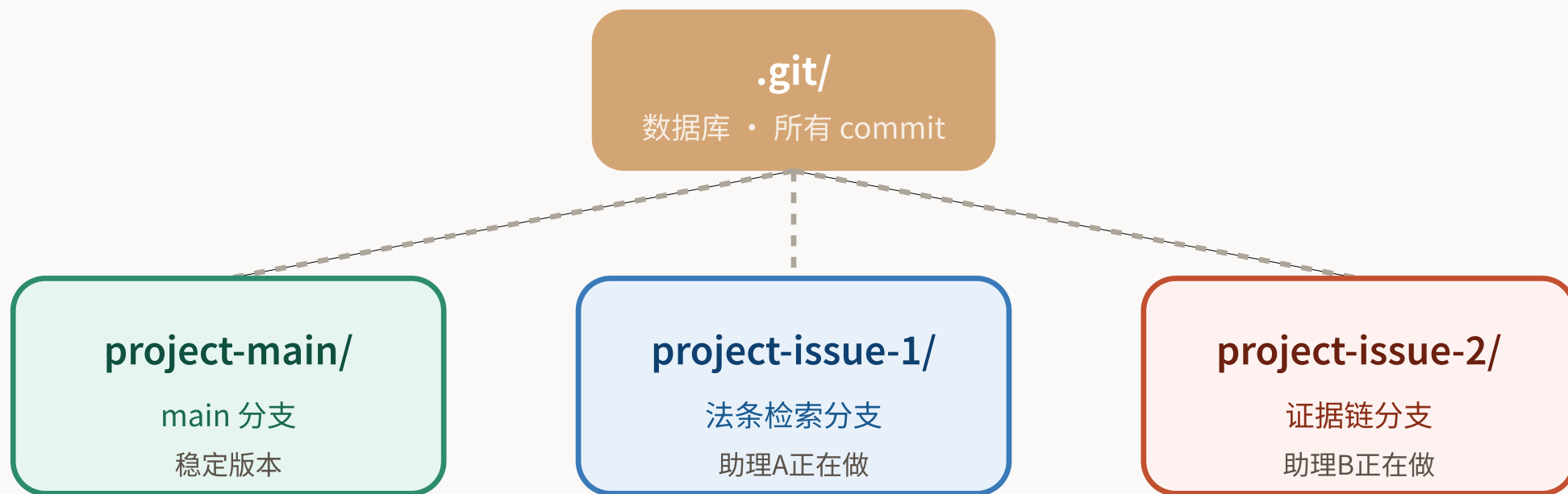
多个分支同时打开

互不干扰，提交共享

用 `git worktree add` 创建

同一栋楼不同房间同时办公

Worktree 示意图



三个文件夹，一套 .git，同时打开互不干扰

```
git worktree add ../project-feature feature-1 # 创建
git worktree list                             # 查看
git worktree remove ../project-feature       # 删除
```

Fork = 在 GitHub/GitLab 上复制一个别人的仓库，变成你自己的。

Clone vs Fork

	Clone	Fork
操作对象	任何仓库	别人的仓库
结果	本地副本	平台上的副本
能推送吗	看权限	一定能
用途	下载代码	参与开源

什么时候用？

给开源项目提改进： Fork → 修改 → 提 PR 给原项目

基于别人的项目做自己的事： Fork → 独立发展

像什么？ 复印一本书，在复印件上随意修改。原书不变，复印件随你改。

Issue = 一个待办事项，带追踪功能。不是"问题"，是"任务"（虽然英文直译是"问题"）

一个 Issue 包含

标题 — 简洁描述任务

描述 — 详细说明目标和上下文

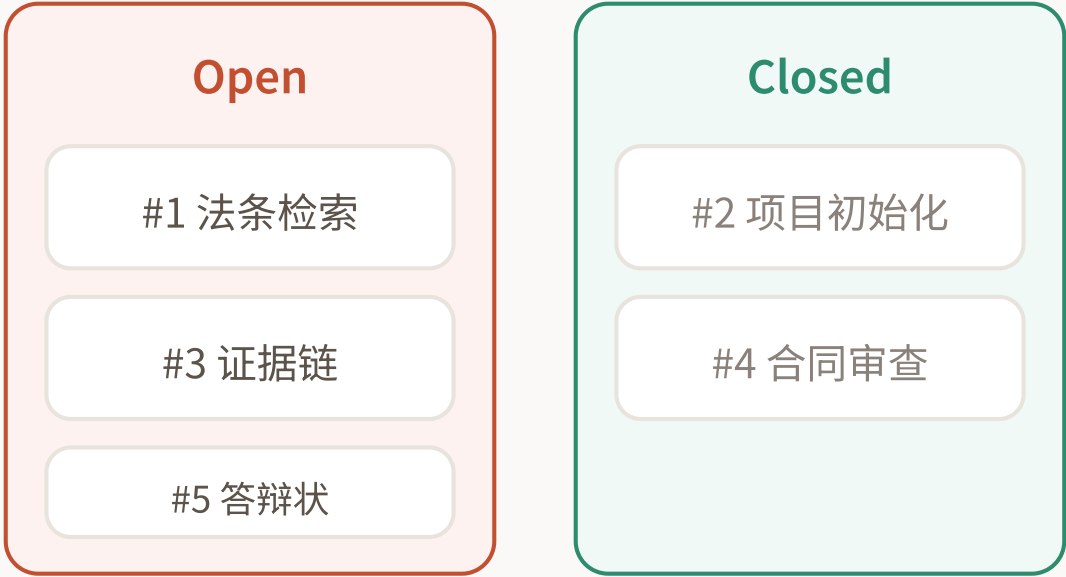
标签 — 如 #法条检索 #高优先级

负责人 — @某人

状态 — Open / Closed

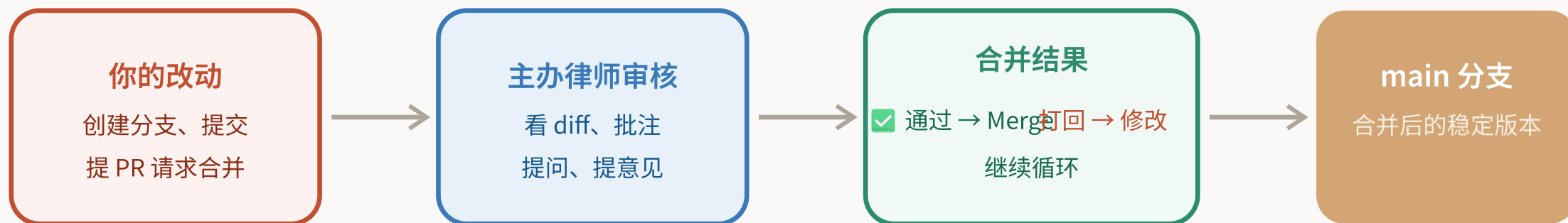
评论历史 — 讨论和更新记录

Issue 列表 = 项目仪表盘



像什么？ 白板上的便利贴 + 聊天记录 + 状态追踪。

Pull Request (PR) = 请求合并代码，等待审核。 Merge Request (MR) = GitLab 的叫法，和 PR 是一回事。



像什么？ PR = 论文投稿。写完论文 → 投稿 → 审稿人评审 → 大修/小修/接收。

.gitignore = 告诉 Git "这些文件别追踪"。 为什么需要？临时文件、依赖、敏感信息、编译产物都不该进 Git。

示例

```
# 忽略所有 .tmp 文件
*.tmp

# 忽略整个文件夹
node_modules/
dist/

# 忽略敏感配置
.env
config.json

# 不忽略某个文件（例外）
!important.tmp
```

法律项目注意

不要把客户资料、证据原件、敏感文档提交到 Git。

```
# 法律项目 .gitignore
*.pdf
*.docx
证据材料/
客户信息/

# 但保留索引文件
!materials-index.md
```

已提交过的文件：再加 .gitignore 无效。需要先 `git rm --cached 文件名`

11 Git 追踪原理

Git 对文件只有两种态度：已追踪 / 未追踪。新文件默认未追踪，必须 `git add` 告诉 Git "我要管"。



仓库空间会爆炸吗？

文本文件（代码、Markdown）

Git 用 delta 压缩，只存变化

100 次修改 $\approx$ 原始大小 + 10%

几乎不占空间

✅ 很省，不是问题

二进制文件（PDF、图片）

Git 无法压缩差异

每次修改存完整副本

3次修改 = 3倍大小

⚠️ 爆炸，迅速膨胀

务实方案：文本（研究、论证、草稿）→ Git 管理。二进制（PDF、图片、视频）→ 外部存储，Git 里只存索引。

实战：创建第一个仓库

目标：把一个案件文件夹变成 Git 仓库

1. 进入文件夹

```
cd ~/我的案件
```

2. 初始化 Git

```
git init
```

3. 创建项目说明

```
echo "# 合同纠纷案" > README.md
```

4. 添加到暂存区

```
git add README.md
```

5. 提交（拍第一张快照）

```
git commit -m "初始提交：创建案件项目"
```

6. 查看状态和历史

```
git status
```

```
git log --oneline
```

实战：文件修改与提交

添加新文件

```
# 1. 创建文件
echo "## 相关法条" > 法条检索.md

# 2. 查看状态
git status
# → Untracked files: 法条检索.md

# 3. 添加到暂存区
git add 法条检索.md

# 4. 提交
git commit -m "添加基础法条检索"
```

修改已有文件

```
# 编辑文件后
git diff
# 绿色：增加 | 红色：删除

# 添加并提交
git add 法条检索.md
git commit -m "补充违约金法条"
```

git add 的作用：告诉 Git"我要管这个文件"，暂存区像购物车，选好商品后统一结账(commit)。

实战：创建分支试想法

场景：想尝试一个新论证角度，但不想影响主线

```
# 当前在 main 分支
git branch
# * main

# 创建并切换到新分支
git checkout -b 新论证角度

# 修改文件、提交
echo "## 新论证思路" >> 答辩状.md
git add 答辩状.md
git commit -m "尝试新论证角度"

# 现在有两个分支，可以随意切换
git checkout main      # 答辩状恢复原样
git checkout 新论证角度 # 新内容又回来了
```

分支让你可以：大胆尝试，不行就扔；多个思路并行；随时对比哪个更好。

实战：合并分支

场景：新论证角度效果不错，合并到主线

```
# 1. 切换到目标分支
git checkout main

# 2. 合并
git merge 新论证角度

# 3. 如果有冲突, Git 会标记
# 手动解决后:
git add 文件名
git commit -m "解决合并冲突"

# 4. 查看历史
git log --oneline --graph
```

合并冲突：是因为同一处被两个人改了。Git 无法判断谁对谁错，需要你决定保留什么。

实战：用 Issue 管理任务

场景：一个复杂诉讼，拆成多个任务

#1 检索相关法条

标签：#法条检索 #高优先级

负责人：@助理A

状态：Open

#2 整理证据链

标签：#证据 #高优先级

负责人：@助理B

状态：Open

#3 起草答辩状

标签：#文书 #中优先级

负责人：@主办律师

状态：Open（依赖 #1 #2）

#4 准备庭审材料

标签：#庭审 #低优先级

负责人：待定

状态：Open

实战：Issue + Branch + PR 完整流程

1

建 Issue: "答辩点3: 损失构成分析"

2

```
git checkout -b issue-003-损失构成
```

3

干活: 检索、论证, 写材料 → `git add .` → `git commit -m "完成损失构成分析"`

4

```
git push origin issue-003-损失构成
```

5

提 PR: 网页上点击 "Create Pull Request", 写清楚改了什么、依据、还有什么不确定

6

审核: 主办律师看 diff, 批注、提问

7

修改: 按反馈调整 → `git push`

8

合并 + 关闭 Issue

实战：Worktree 并行处理多任务

场景：同时处理三个任务，不想来回切换

```
# 主仓库在 ~/litigation
cd ~/litigation

# 创建三个 worktree
git worktree add ~/litigation-main main
git worktree add ~/litigation-issue-001 issue-001-法条检索
git worktree add ~/litigation-issue-002 issue-002-证据链

# 现在你有三个文件夹，可以同时打开工作
# 每个文件夹独立，互不干扰，提交都回到同一个仓库

git worktree list # 查看
git worktree remove ~/litigation-issue-001 # 用完删除
```

优势：不用来回切换分支，同时保持多个任务的上下文。

实战：三台电脑同步

架构：一台电脑当 Git 服务器



电脑A设置:

```
mkdir ~/git-repos && cd ~/git-repos
```

```
git init --bare litigation.git
```

电脑B、C连接:

```
git clone ssh://电脑A的IP/~/git-repos/litigation.git
```

避坑：二进制文件处理

方案一：外部存储 + 索引（推荐）

仓库 (Git 管理)

```
├─ 案件概述.md
├─ 证据索引.md    ← 只存清单
├   └─ 证据1.pdf → ../materials/
└─ 答辩状.md
```

外部存储

```
└─ materials/
   ├─ 证据1.pdf (50MB)
   └─ 证据2.pdf (30MB)
```

方案二：Git LFS（大文件存储）

```
# 安装 Git LFS
git lfs install

# 追踪 PDF 文件
git lfs track "*.pdf"

# PDF 会存到 LFS 服务器
# .git 里只存指针
```

建议：法律项目优先用方案一，PDF/扫描件存外部，Git 只管文本主线。

工具推荐

工具	用途	适合谁
GitHub Desktop	可视化 Git 操作	零基础入门
Fork	高级 Git 客户端，看分支图	需要详细操作
GitKraken	可视化 + 团队协作	团队使用
VS Code	编辑器 + Git 集成	写 Markdown/代码
Obsidian	本地笔记 + Git 插件	知识管理

建议：先学命令行理解原理，再用图形界面提高效率。命令行是内功，图形界面是轻功。

命令速查表

仓库操作

```
git init          # 初始化
git clone <地址>   # 克隆
git status        # 查看状态
git log --oneline  # 查看历史
```

提交操作

```
git add <文件>    # 添加到暂存
git add .          # 添加所有
git commit -m "消息" # 提交
git commit -am "消息" # 添加并提交
```

分支操作

```
git branch        # 查看分支
git branch <名字>  # 创建分支
git checkout <分支> # 切换
git checkout -b <名> # 创建并切换
git merge <分支>   # 合并
```

远程与 Worktree

```
git push origin <分支> # 推送
git pull               # 拉取
git worktree list       # 查看
git worktree add <路径> <分支>
```

总结：核心转变

以前	以后
文件命名"最终版1/2/3"	Commit 历史，每步可追溯
微信/邮件零散沟通	Issue + PR 结构化协作
主办律师什么都得做	任务拆分，审核决策
AI 是临时工	AI 是项目成员，可领任务

记住三个核心

- **Commit** = 快照，不是差异
- **Branch** = 指针，不是复制
- **Issue + PR** = 任务提出 → 执行 → 审核 → 合并

最后建议：不要试图一次学会所有。先创建一个仓库，做几个 commit，建几个 branch，提一个 PR。**动手比看十遍都强。**

学习资源



那篇启发文章

《Git issue + PR：律师的下一代协作方式》

作者：杨卫薪

示例仓库：github.com/cat-xierluo/litigation-ai-demo



官方文档

Git 官方：git-scm.com/doc

GitHub Docs：docs.github.com

GitLab Docs：docs.gitlab.com

推荐书籍：《Pro Git》（免费在线版）· 《Git 权威指南》

动手试试吧

创建你的第一个仓库

开始了，就成功了一半

```
mkdir my-project  
cd my-project  
git init  
echo "# 我的第一个项目" > README.md  
git add README.md  
git commit -m "初始提交"
```

《GitHub 工作法：律师的协作新方式》